| | |
|---|---|
| Experiment No: | 3 |
| Experiment Title: | Regression Analysis using Linear and Regularized Models |
| Name: | NAVINKUMAR S |
| RegNo: | 3122247001039 |
| Sem and Academic Year: | Vth sem (2026-2027) |
| Submission Date | 02-08-2026 |

# 1 Aim and Objective

To predict the loan amount that will be sanctioned using Linear Regression and three regularized versions of it – Ridge, Lasso and Elastic Net. Along with building these models, the experiment also tunes the regularization strength of each model, compares all four models using standard regression metrics, and studies how each model behaves in terms of overfitting, underfitting, and the bias–variance trade-off.

# 2 Dataset Description

| | |
|---|---|
| Dataset Name | Loan Amount Prediction Dataset |
| Dataset Source | Kaggle – https://www.kaggle.com/datasets/phileinsophos/predict-loan-amount-data |
| Target Variable | Loan Sanction Amount (USD) – a continuous number, which is why this is a regression problem |
| Training Rows | 30,000 |
| Test Rows (unlabeled set) | 20,000 |
| Total Columns | 24 (23 features + target) |
| Numerical Columns | 13 |
| Categorical Columns | 11 |
| Columns with Missing Values | 11 |
| Duplicate Rows | 0 |
| Highly Correlated Feature Pairs | 2 |

The dataset holds information about loan applicants – their income, profession, employment type, credit score, existing loans, and details about the property being financed. Eleven of the columns had missing values, including 'Gender', 'Income (USD)', 'Income Stability', 'Type of Employment', 'Current Loan Expenses (USD)', 'Dependents', 'Credit Score', 'Has Active Credit Card', 'Property Age', 'Property Location', and even the target column itself, 'Loan Sanction Amount (USD)'. No duplicate rows were found, so nothing needed to be dropped on that front.

# 3 Preprocessing Steps

- **Missing value imputation:** Categorical columns (Gender, Income Stability, Type of Employment, Has Active Credit Card, Property Location) were filled with the **mode**. Money-related numeric columns (Income, Loan Amount Request, Current

Loan Expenses, Dependents, Property Age, Property Price, and the target itself) were filled with the **median**, since these columns are skewed. Credit Score was filled with the **mean**, since it behaves more like an evenly spread score.

- **Placeholder values:** A few columns used -999 as a stand-in for a missing value instead of leaving it blank ('Current Loan Expenses (USD)', 'Loan Sanction Amount (USD)', 'Co-Applicant', 'Property Price'), so these were first converted to a real missing value before being filled in. The test set also stored some missing values as the text '?' in 'Co-Applicant' and 'Property Price', so both columns were converted to numbers first so that the -999 step and the median fill would work correctly.
- **Encoding categorical variables:** 'Income Stability' is a simple two-value column (Low/High), so it was mapped directly to 0/1. The remaining text columns (Customer ID, Name, Gender, Profession, Type of Employment, Location, Expense Type 1, Expense Type 2, Has Active Credit Card, Property Location) were converted to numbers using a LabelEncoder, since regression models can only work with numbers, not text.
- **Feature scaling:** All features were standardized using StandardScaler before training. This step is more important for Ridge, Lasso and Elastic Net, because these models add a penalty based on the size of each coefficient – if features were left on very different scales, that penalty would unfairly hurt features that are naturally measured in larger numbers (like income) compared to smaller ones (like number of dependents).
- **Train-test split:** The training data was split into a 75% training portion and a 25% hold-out portion, so that every model could be judged on data it had not seen while being trained.

# 4 Implementation Details

Four regression models were implemented:

- **Linear Regression** – trained with no regularization, used as the baseline.
- **Ridge Regression** – tuned over $\alpha \in \{0.01, 0.1, 1, 10, 100\}$.
- **Lasso Regression** – tuned over $\alpha \in \{0.001, 0.01, 0.1, 1, 10\}$.
- **Elastic Net Regression** – tuned over $\alpha \in \{0.01, 0.1, 1, 10\}$ and l1 ratio $\in \{0.2, 0.5, 0.8\}$.

All three regularized models were tuned using GridSearchCV with 5-fold cross validation, optimizing the R2 score. Every model was then evaluated on the test split using Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), R2 score, and training time.

# 5 Visualizations

## Target Variable Distribution

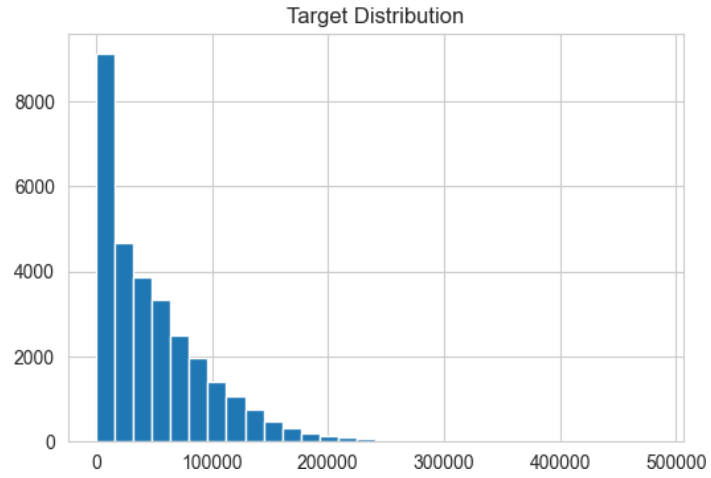


Figure 1: Distribution of Loan Sanction Amount (USD)

**Inference:** The loan amount sanctioned is not spread evenly – most applicants are sanctioned smaller amounts, with a smaller number of applicants receiving much larger loans, giving the distribution a long right tail.

## Feature vs Target Scatter Plots

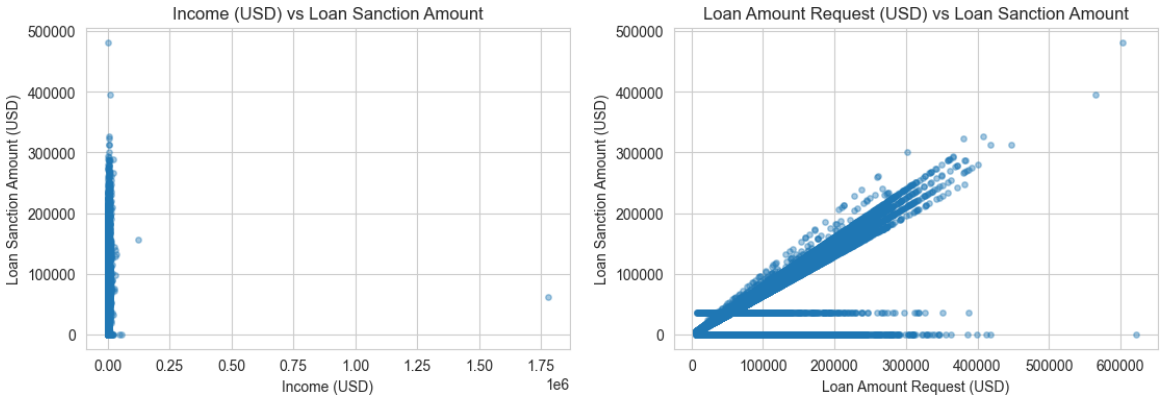


Figure 2: Income (USD) and Loan Amount Request (USD) vs Loan Sanction Amount

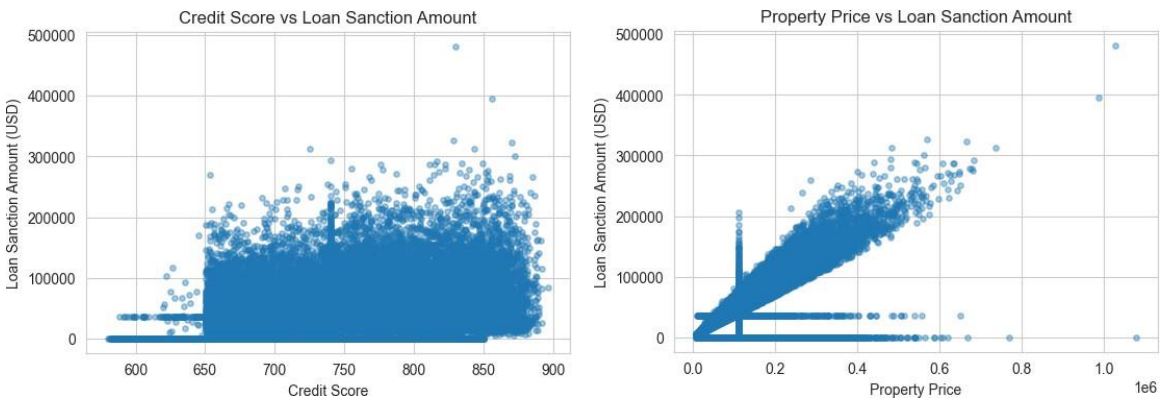


Figure 3: Credit Score and Property Price vs Loan Sanction Amount

**Inference:** Loan Amount Request shows the clearest straight-line relationship with the sanctioned amount, which makes sense – the amount sanctioned is naturally related to the amount requested. Income and Credit Score show a weaker, noisier upward trend, while Property Price does not show a strong relationship on its own.

## Predicted vs Actual and Residual Plots

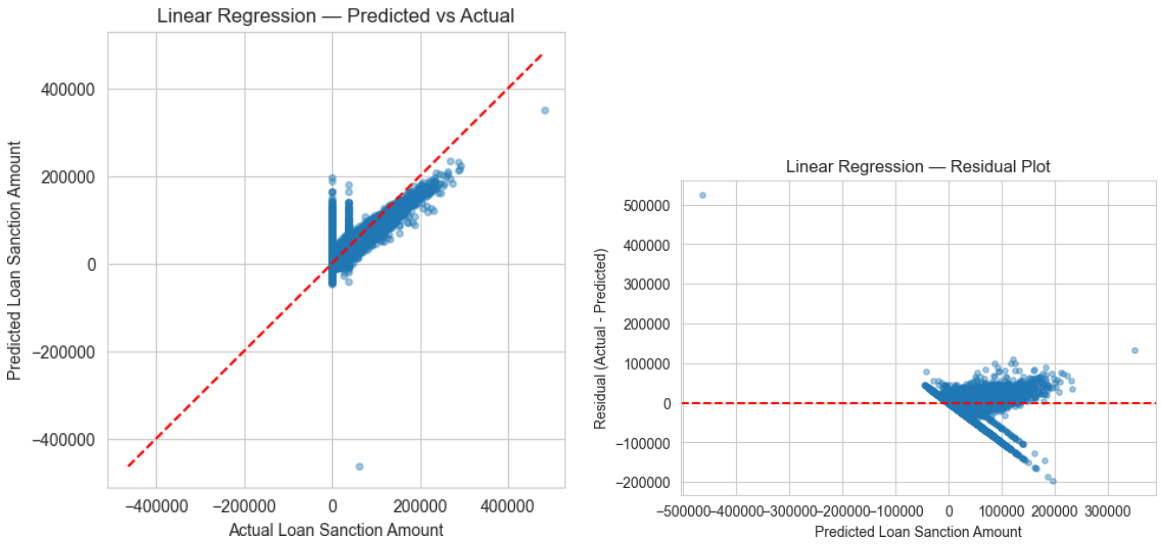


Figure 4: Linear Regression – Predicted vs Actual and Residual Plot

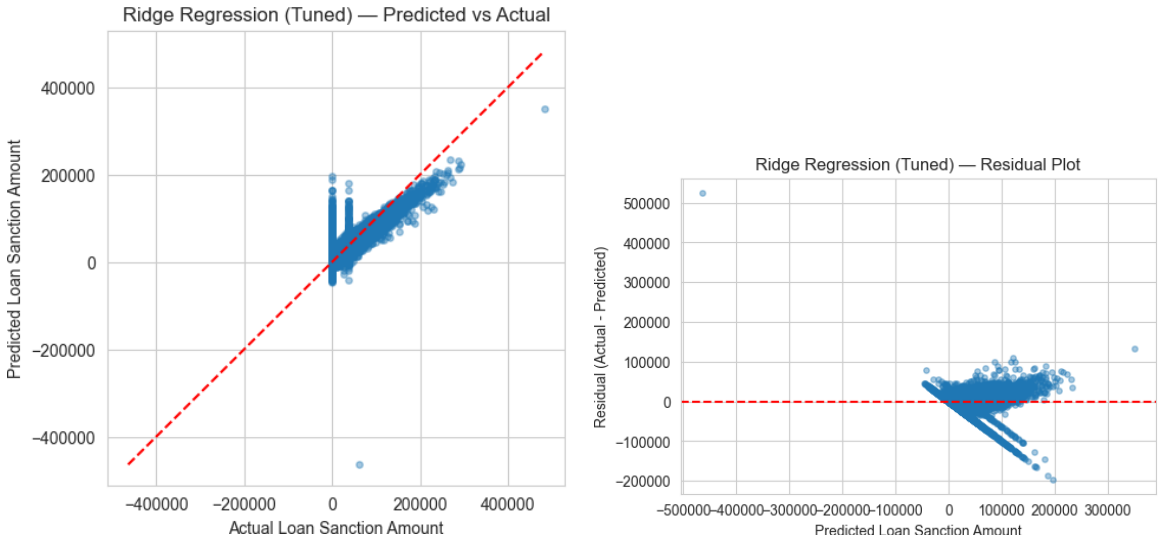


Figure 5: Ridge Regression (Tuned) – Predicted vs Actual and Residual Plot

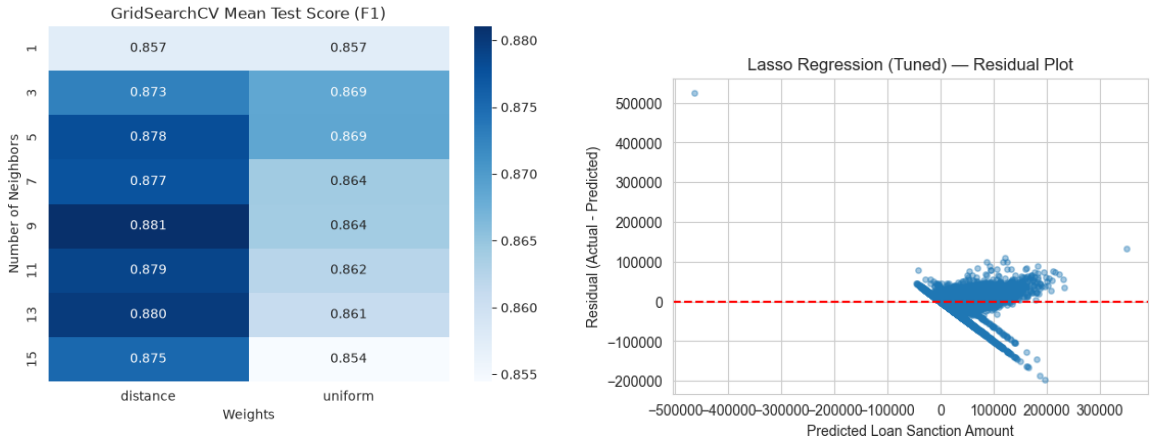


Figure 6: Lasso Regression (Tuned) – Predicted vs Actual and Residual Plot

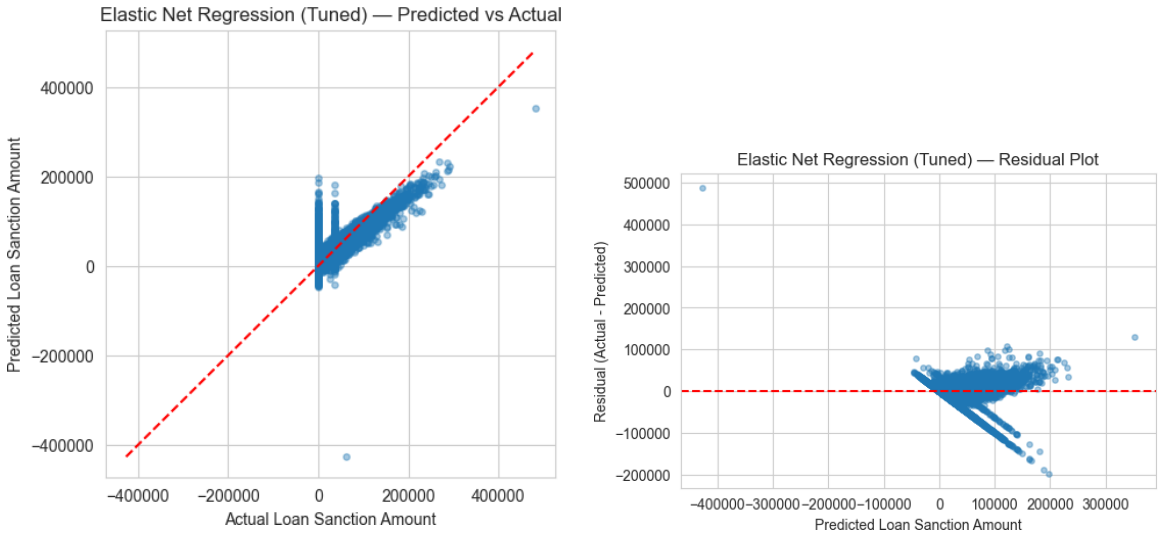


Figure 7: Elastic Net Regression (Tuned) – Predicted vs Actual and Residual Plot

**Inference:** All four models show a similar pattern – the predicted points loosely follow the red reference line but with a wide spread, meaning the models capture the general trend in loan amounts but are far from perfectly accurate on individual applicants. The residual plots show the errors scattered fairly evenly above and below zero without any obvious curve or pattern, which means the models are not missing some obvious non-linear relationship in the data.

## Coefficient Comparison

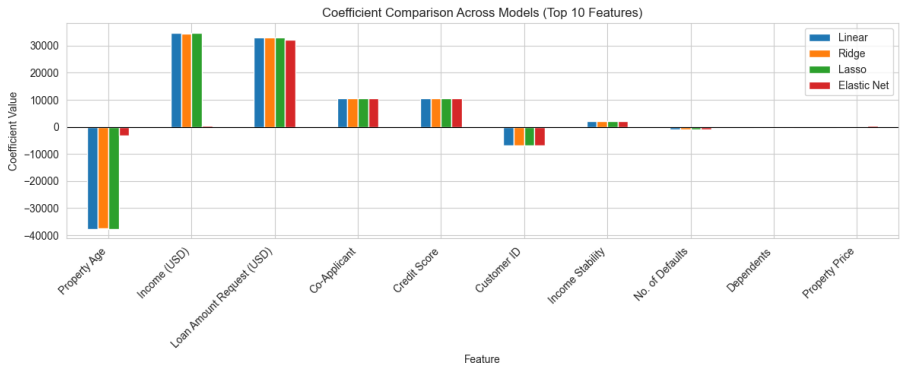


Figure 8: Coefficient Comparison Across Models (Top 10 Features by Linear Regression)

**Inference:** Property Age stands out the most – its coefficient is very large and negative under plain Linear Regression, but shrinks dramatically under Elastic Net. This indicates that Linear Regression was leaning heavily on a feature that is not actually very reliable (Property Age had a large number of missing values that had to be filled in), and the regularized models pulled that coefficient back down to a more reasonable size. Income, Loan Amount Request, Credit Score and Co-Applicant stay large and positive across all four models, suggesting these are, important predictors of the loan amount.

## Training vs Validation Error (Bias–Variance Trade-off)

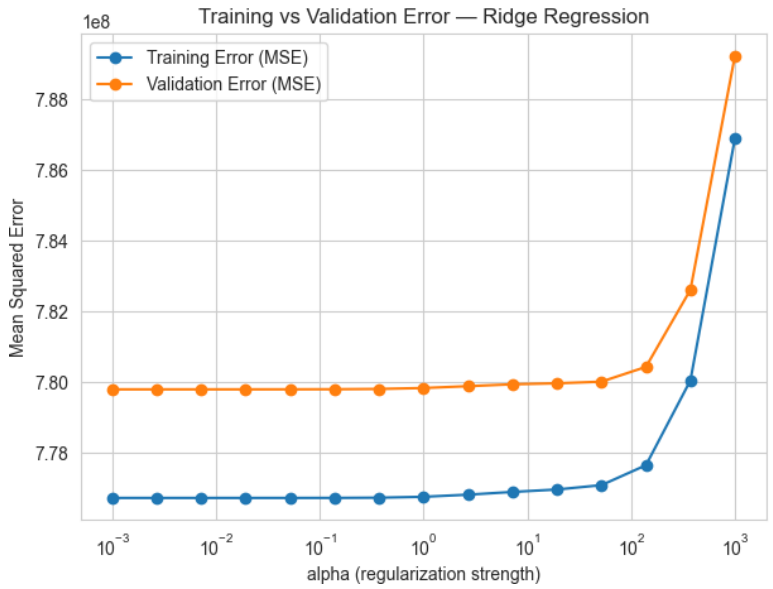


Figure 9: Training Error vs Validation Error across different Ridge alpha values

**Inference:** At very small alpha values, training and validation error are both already close to their lowest point and close to each other, so the model is not overfitting even with almost no regularization. As alpha increases further , both curves rise together, which makes underfitting, once the penalty becomes too strong, the model is forced to be too simple to capture the real relationship in the data, and it gets worse on both the data it was trained on and the data it was tested on.

## Test Set Performance Comparison


Figure 10: Test Set R2 Score Comparison Across All Four Models

**Inference:** All four models land at almost the same R2 score, with Elastic Net coming out very slightly ahead. This matches the earlier finding that regularization barely changed

anything here – the best alpha values chosen by GridSearchCV were all very small, meaning plain Linear Regression was already close to as good as it could get on this dataset.

# 6 Performance Tables

### Table 1: Hyperparameter Tuning Summary

| Model | Search Method | Best Parameters | Best CV R2 |
|---|---|---|---|
| Ridge Regression | Grid Search | $\alpha = 0.01$ | 0.6590 |
| Lasso Regression | Grid Search | $\alpha = 0.001$ | 0.6590 |
| Elastic Net Regression | Grid Search | $\alpha = 0.01$, l1_ratio = 0.8 | 0.6589 |

### Table 2: Cross-Validation Performance (K = 5, on the training set)

| Model | MAE | MSE | RMSE | R2 |
|---|---|---|---|---|
| Linear Regression | 19098.55 | $7.798 \times 10^8$ | 27924.93 | 0.6590 |
| Ridge Regression | 19098.52 | $7.798 \times 10^8$ | 27924.93 | 0.6590 |
| Lasso Regression | 19098.55 | $7.798 \times 10^8$ | 27924.93 | 0.6590 |
| Elastic Net Regression | 19107.71 | $7.800 \times 10^8$ | 27928.29 | 0.6589 |

### Table 3: Test Set Performance

| Model | MAE | MSE | RMSE | R2 |
|---|---|---|---|---|
| Linear Regression | 18749.14 | $7.778 \times 10^8$ | 27889.80 | 0.6532 |
| Ridge Regression (Tuned) | 18749.07 | $7.778 \times 10^8$ | 27889.59 | 0.6532 |
| Lasso Regression (Tuned) | 18749.14 | $7.778 \times 10^8$ | 27889.79 | 0.6532 |
| Elastic Net Regression (Tuned) | 18744.99 | $7.729 \times 10^8$ | 27800.90 | **0.6554** |

Training time followed the expected order: Linear Regression was fastest (0.031 seconds, since it does not search over any hyperparameters), followed by Ridge (3.67 seconds), Elastic Net (4.51 seconds), and Lasso was the slowest to tune (41.93 seconds), since Lasso's optimization algorithm takes more iterations to converge than Ridge's does.

### Table 4: Coefficient Comparison (Top Features)

| Feature | Linear | Ridge | Lasso | Elastic Net |
|---|---|---|---|---|
| Property Age | −37616.35 | −37458.38 | −37606.51 | −3121.88 |
| Income (USD) | 34643.72 | 34486.16 | 34633.91 | 360.63 |
| Loan Amount Request (USD) | 32953.66 | 32953.51 | 32953.63 | 32097.26 |
| Co-Applicant | 10571.56 | 10571.54 | 10571.55 | 10547.65 |
| Credit Score | 10512.97 | 10512.95 | 10512.97 | 10499.76 |
| Customer ID | −6937.84 | −6937.86 | −6937.84 | −6931.99 |
| No. of Defaults | −818.76 | −818.78 | −818.76 | −820.42 |

Lasso, at its tuned alpha of 0.001, did not drop any features completely – it set 0 out of 23 coefficients to exactly zero. This is expected, since such a small alpha applies only a very light penalty, barely different from plain Linear Regression.

# 7 Overfitting and Underfitting Analysis

Looking at the gap between the cross-validation R2 (Table 2, around 0.659 for all models) and the test R2 (Table 3, around 0.653–0.655), the drop is small – only about 0.005 to 0.006. This small, consistent gap suggests that Linear Regression was not overfitting badly to begin with, which is also why regularization barely changed its performance. GridSearchCV picked very small alpha values for both Ridge (0.01) and Lasso (0.001), which is the search essentially saying ”not much regularization is needed here.” The training-vs-validation error plot supports this too – training and validation error start off already close together at small alpha, and only grow apart once alpha is pushed high enough to visibly underfit the data. With 30,000 training rows and only 23 features, there is enough data relative to model complexity that plain Linear Regression does not have much chances to overfit.

# 8 Bias–Variance Analysis

Linear Regression has the lowest bias because it does not penalise the coefficients, but this also makes it more sensitive to noisy features. In this dataset, Property Age has many imputed values, and its coefficient becomes very large (-37616), indicating possible overfitting. Ridge Regression shows similar behaviour because its penalty is very small. Elastic Net shrinks the coefficients, making the model less affected by noisy features, and achieves the highest test $R^2$ (0.6554), although the improvement is only slight.

# 9 Observations and Conclusion

All four models—Linear, Ridge, Lasso, and Elastic Net—performed very similarly, with test $R^2$ scores between 0.653 and 0.655. GridSearchCV selected very small regularisation values for Ridge and Lasso, indicating that Linear Regression already fit the data well and overfitting was not a major issue. Elastic Net achieved the highest test $R^2$ and produced more stable coefficients by reducing the influence of Property Age, which contained many missing values. One limitation is that identifier columns such as Customer ID and Name were label encoded and included in the model. Since these values do not have meaningful numerical relationships, their coefficients should not be interpreted, and these columns should ideally be removed before training. Overall, Elastic Net is the preferred model because it provides the best accuracy while being more robust to noisy features.

# 10 References

1. Scikit-learn Developers, “Linear Models,” https://scikit-learn.org/stable/modules/linear_model.html.

2. Scikit-learn Developers, “Hyperparameter Optimization,” https://scikit-learn.org/stable/modules/grid_search.html.

3. Kaggle, “Predict Loan Amount Data.” https://www.kaggle.com/datasets/phileinsophos/predict-loan-amount-data.